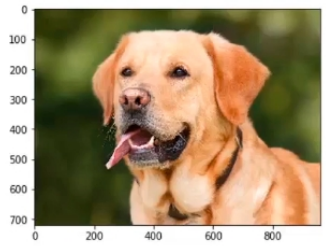
We can consider Convolutional Neural Networks, or CNNs, as feature extractors that help to extract features from images.

In a simple neural network, we convert a 3-dimensional image to a single dimension. Following image is a 1-D representation whereas the second one is a 2-D representation of the same image.

Spatial Orientation

Here, the orientation of the images has been changed but we are unable to identify it by looking at the 1-D representation.

This is the problem with artificial neural networks – they lose spatial orientation



Large number of parameters

Another problem with neural networks is the large number of parameters. If our image has a size of 28*28*3 – so the parameters here will be 2,352. What if we have an image of size 224*224*3? The number of parameters here will be 150,528. And these parameters will only increase as we increase the number of hidden layers.

So, the two major disadvantages of using artificial neural networks are:

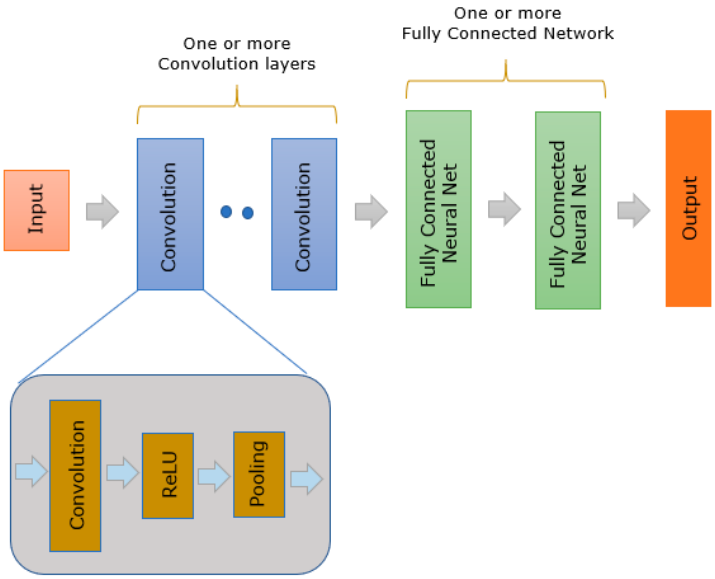
1. Loses spatial orientation of the image
2. The number of parameters increases drastically

How can we preserve the spatial orientation as well as reduce the learnable parameters. CNNs help to extract features from the images which may be helpful in classifying the objects in that image. It starts by extracting low

dimensional features (like edges) from the image, and then some high dimensional features like the shapes. We use filters to extract features from the images and Pooling techniques to reduce the number of learnable parameters.

CNN (Convolutional Neural Network) Architecture:

CNN is made up two large groups of components :: Convolutional Layer and Fully Connected Network layer.



Convolutional Layer : This performs the function of pattern detection from the input image. The pattern detection by convolution layer is proved working very well even when the size of the target image varies (changes), position and rotation of the image changes. It would be easy to realize that this would be much better than hand-written rule based pattern detection. In reality, it works better than fully connected network. It is said that we may need much more neurons (i.e, much more number of weights) for a fully connected network to perform the same level as convolutional layers. On top of it, Fully Connected Network may not be as robust as convolutional layer in terms of finding patterns from geometrically transformed image (like different size, rotated, translated).

Fully Connected Network : This performs the classification of the image combining a different types of patterns detected by Convolutional Layers.

The result of the convolution goes through a special function called ReLU(Rectified Linear Unit). ReLU is a activation function(transfer function) for Convolution layer. The output of ReLU data goes through another process called Pooling. Pooling is a kind of Sampling method to reduce the number of data without loosing the critical nature of the convolved data.

After the convolution layer in CNN, there exist one or more of fully connected neural network. Fully Connected Network is a simple feedforward network.

In CNN, the exact type of features to be detected is determined by the network itself during the learning process. The element values for each of the kernel is not fixed/predefined and is set randomly and gets updated (changes) during learning process.

CNN has become one of the most popular neural networks since it showed such a great performance on image classification. Most of neural network that is used for image recognition or classification is based on CNN.

Exercise Questions:

1. Implement backpropagation for a Multilayer Perceptron (MLP) trained on a regression dataset using only matrix operations. You are required to:
    a. Implement the forward pass using matrix multiplication and vectorized activation functions.
    b. Compute the loss function (Mean Squared Error) in matrix form.
    c. Derive and implement the backward pass using the chain rule expressed entirely as matrix operations.
    d. Update the weights and biases using gradient descent in a fully vectorized manner (no explicit loops over neurons).
    e. Train the MLP on a given regression dataset and report the training loss over epochs.
2. Implement a convolution operation for a sample image of shape (H=6, W=6, C=1) with a random kernel of size (3,3) using torch.nn.functional.conv2d.

```
import torch

import torch.nn.functional as F
```

```
image = torch.rand(6,6)
print("image=", image)
#Add a new dimension along 0th dimension
#i.e. (6,6) becomes (1,6,6). This is because
#pytorch expects the input to conv2D as 4d tensor
image = image.unsqueeze(dim=0)
print("image.shape=", image.shape)
image = image.unsqueeze(dim=0)
print("image.shape=", image.shape)
print("image=", image)
kernel = torch.ones(3,3)
#kernel = torch.rand(3,3)
print("kernel=", kernel)
kernel = kernel.unsqueeze(dim=0)
kernel = kernel.unsqueeze(dim=0)
#Perform the convolution
outimage = F.conv2d(image, kernel, stride=1, padding=0)
print("outimage=", outimage)
```

What is the dimension of the output image? Apply, various values for parameter *stride=1*
and note the change in the dimension of the output image. Arrive at an equation for the output image size with respect to the kernel size and stride and verify your answer with code. Now, repeat the exercise by changing padding parameter. Obtain a formula using kernel, stride, and padding to get the output image size. What is the total number of parameters in your network? Verify with code.

3. Apply torch.nn.Conv2d to the input image of Qn 1 with out-channel=3 and observe the output. Implement the equivalent of torch.nn.Conv2d using the torch.nn.functional.conv2D to get the same output. You may ignore bias.

4. Implement CNN for classifying digits in MNIST dataset using PyTorch. Display the classification accuracy in the form of a Confusion matrix. Verify the number of learnable parameters in the model.

   Training a CNN on an image dataset is similar to training a basic multi-layer feed-forward network on numerical data as outlined below.

   Define model architecture
   Load dataset from disk
   Loop over epochs and batches
   Make predictions and compute loss
   Properly zero our gradient, perform backpropagation, and update model parameters

```
class CNNClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        #self.w = torch.nn.Parameter(torch.rand([1]))
        #self.b = torch.nn.Parameter(torch.rand([1]))
        self.net =nn.Sequential(nn.Conv2d(1,64,kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2,2), stride=2),
                                nn.Conv2d(64, 128, kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2,2), stride=2),
                                nn.Conv2d(128, 64, kernel_size=3),
                                nn.ReLU(),
                                nn.MaxPool2d((2, 2), stride=2)
        )
        self.classification_head = nn.Sequential(nn.Linear(64, 20, bias=True),
                                nn.ReLU(),
                                nn.Linear(20, 10, bias=True))
    def forward(self, x):
        features = self.net(x)
        return self.classification_head(features.view(batch_size,-1))
```

5. Modify CNN of Qn. 4 to reduce the number of parameters in the network. Draw a plot of percentage drop in parameters vs accuracy.
6. Design CNN to classify images using Fashion MNIST dataset. Display the classification accuracy in the form of a Confusion matrix. Verify the number of learnable parameters in the model.